Group Assignment: build and secure a tested REST API

Course: Testing and Authentication
Groups: 1–3 students
Time: 7 days
Submission: GitHub repo link

Overview

You are going to build a small but complete REST API that is **tested**, **authenticated**, and **automatically verified by a CI pipeline**. Each piece connects directly to what you have learned in this course : TDD, unit and integration testing, GitHub Actions, Firebase or Auth0, and security best practices.

You will work as a team or alone if needed. If you work in a team, divide the work, and submit one shared GitHub repository.

What you are building

A **Gym Review API** with the following routes:

Method	Route	Access
GET	`/gyms`	Public
GET	`/gyms/:id`	Public
POST	`/gyms`	Protected (must be logged in)
POST	`/gyms/:id/reviews`	Protected (must be logged in)
GET	`/profile`	Protected (must be logged in)

You choose the database (for example PostgreSQL or MongoDB, but an in-memory array is also fine for this assignment. The focus is on **testing and authentication**, not database design.

Requirements

1. Project setup

- One shared GitHub repository with a clear folder structure, for example `backend/` and `client/` or `backend/` and `frontend/`
- A `README.md` that explains how to install, run, and test the project, as well as your reflections as a group. Part of your reflection include your implementation choices and security decisions.
- A `.env.example` file showing which environment variables are needed — **no secrets in the code**

- Dependencies installed via `package.json`

---

2. Authentication

Choose **one** of the following and implement it consistently across the project:

Option A: Auth0 with `express-openid-connect`
Use session-based authentication as shown in the tutorial. Protect the `POST` routes and `/profile` with `requiresAuth()`. Set `errorOnRequiredAuth: true` in your config so protected routes return `401` instead of redirecting.

Option B: Firebase with `firebase-admin`
Use token-based authentication as shown in the earlier lesson. Protect the `POST` routes with your `verifyToken` middleware. The client sends a bearer token in the `Authorization` header.

Either way, unauthenticated requests to protected routes must return `401 Unauthorized`. Authenticated requests must return the correct data.

The frontend must include a working **login and logout button** that authenticates with your chosen provider. Users should only see protected content when they are logged in.

---

3. Testing

You must write tests at **two levels**:

Unit tests (Vitest)

Write at least **5 unit tests** that check your UI or utility logic in isolation. Use the fake component pattern from the lessons — no real network calls.

Suggested tests:

- Shows a "not logged in" message when there is no user
- Shows the user's name when logged in
- Hides the protected form when not logged in
- Shows a list of gyms when data is passed in
- Shows an error message when the gym list is empty

Integration tests (Vitest + `node:http`)

Write at least **5 integration tests** that test how multiple parts of your app work together. For example, test that registering a user and then logging in works end to end, without mocking the functions in between.

Suggested tests:

- `GET /gyms` returns `200` and an array
- `GET /gyms/:id` returns `404` for an unknown ID
- `POST /gyms` without a token returns `401`
- `POST /gyms` with a valid session/token returns `201`

- `POST /gyms/:id/reviews` without a token returns `401`

---

4. GitHub Actions pipeline

Create a file at `.github/workflows/test.yml` that:

- Triggers on every push and pull request to `main`
- Installs dependencies
- Runs all your tests
- Shows a pass or fail result in the GitHub Actions tab

Your pipeline must use **GitHub Secrets** for any sensitive values (Auth0 domain, Firebase credentials, etc.) — never hardcode them in the workflow file.

A minimal working pipeline example:

```
name: Run tests
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [22.x]

    steps:
      - uses: actions/checkout@v3
      - name: Use Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix.node-version }}
      - name: Install dependencies
        run: npm install
        working-directory: ./backend
      - name: Run tests
        run: npm test
        working-directory: ./backend
        env:
          SECRET: ${{ secrets.SECRET }}
          CLIENT_ID: ${{ secrets.CLIENT_ID }}
          ISSUER: ${{ secrets.ISSUER }}
```

---

5. Security checklist

Before submitting, go through this list and confirm each point in your written reflection:

- ☐ No secrets or API keys are committed to the repository
- ☐ All sensitive values are in `.env` and listed in `.env.example`
- ☐ Protected routes return `401` for unauthenticated requests (verified by tests)
- ☐ CORS is configured to only allow your frontend origin. Do not use a wildcard `*` but specify the port
- ☐ Tokens are not stored in `localStorage`
- ☐ `withCredentials: true` (or equivalent) is used on all authenticated frontend requests

---

6. README requirements

Your `README.md` must include the following sections:

Setup

- How to clone the repository
- How to install dependencies
- How to configure environment variables (refer to `.env.example`)
- How to run the project locally

Testing

- How to run the tests locally
- A screenshot of passing tests run locally
- A screenshot of the passing GitHub Actions pipeline

Authentication

- Which provider you chose (Autho or Firebase) and why
- How authentication is implemented in your project

Security decisions

- Go through each point in the security checklist and explain not just *what* you did, but *why*. For example, why tokens are not stored in `localStorage`, or why CORS is restricted to a specific origin

Reflections

- What implementation choices did you or your group make and why
- What was challenging, and what would you do differently

Suggested 7-day plan

This is a guide — adapt it to your group's pace.

Day	Focus
1	Set up the repo, agree on folder structure, choose Auth0 or Firebase, write the first failing test together
2	Build the public routes (`GET /gyms`, `GET /gyms/:id`), write integration tests for them
3	Add authentication middleware, protect the `POST` routes, write the 401 tests
4	Write unit tests for the frontend/UI logic
5	Set up the GitHub Actions pipeline, make sure it passes, and add a screenshot of the passing pipeline to the README.
6	Security review and go through the checklist, fix anything missing
7	Write the reflection, clean up the README, add screenshots of passing tests (both locally and in GitHub Actions), final check.

What to submit

A link to your GitHub repository — it must be public or shared with the teacher

---

Grading criteria

Not passed (IG)

- Tests are missing, broken, or do not run
- Protected routes do not return `401` for unauthenticated requests
- No GitHub Actions pipeline, or pipeline is not connected to the tests
- Authentication is not implemented or does not protect routes correctly
- Secrets are committed to the repository
- `.env.example` or `README.md` is missing
- The frontend has no login/logout functionality or does not connect to the API

Passed (G)

- At least 5 working unit tests and 5 working integration tests
- Protected routes correctly return `401` for unauthenticated requests
- A working GitHub Actions pipeline that runs tests on push
- Authentication implemented with Firebase or Auth0
- No secrets in the repository
- A complete `.env.example` and a clear `README.md`
- The frontend includes a working login/logout button and displays protected content only when logged in

---

Well passed (VG)

- Tests are well named, well structured, and cover both success and failure cases
- Protected routes return `401` and are verified by clearly named tests
- The pipeline includes a clear test report or summary in the GitHub Actions output
- Authentication is fully integrated and the group can explain their implementation choices
- No secrets in the repository and sensitive values are handled deliberately
- A complete .env.example and a clear README.md with a clear understanding of *why* each security decision was made and not just *what* was done
- The frontend handles loading states and errors gracefully, and uses `withCredentials: true` or equivalent on all authenticated requests